

使用生成式 AI 導入邊緣 運算裝置的測試與服務

雙協定混合架構的 RESTful API 自動化測試系統

Dual-Protocol Hybrid Architecture for Automated RESTful API Testing

研究生：113AB8037 鄭亦恩

指導教授：彭祖乙 教授

資訊與財金管理系碩士班

中華民國 114 年 12 月 17 日

Overview

01 Abstract

02 Introductio

03 Problem

04 Objective

05 Methodology

06 Result

07 Conclusion

08 Reference

Abstract

隨著 CI/CD 與微服務架構的普及，RESTful API 測試面臨**語義一致性不足**與**測試序列長度有限**的挑戰。

- 本研究提出一套基於 **A2A (Agent-to-Agent)** 與 **MCP (Model Context Protocol)** 雙協定整合的創新多代理系統，用於實現高效能的 RESTful API 自動化黑箱測試。
- 系統採用 **AutoGen 框架** 作為 A2A Host 進行任務協調，並透過 **FastMCP Server (LangChain 支持)** 實現語義增強與上下文管理。整合 RESTLess 的混合優化策略，包括語義增強與權重式渲染順序優化，顯著提升測試腳本的準確性與覆蓋率。

Introduction

軟體開發的結構性矛盾

速度 vs. 品質落差：開發節奏加快，但品質驗證機制相對滯後

CI/CD 帶來的挑戰

- 高頻率更新導致狀態爆炸
- 測試腳本難以匹配迭代節奏
- 文件與系統狀態知識漂移
- 人工測試成本高昂

RESTful API 測試痛點

- 缺乏語義一致性
- 測試序列長度不足
- 難以處理複雜依賴關係
- End-to-End 驗證困難

Problem

傳統 RESTful API 測試的核心問題

問題 1：語義不足 (Semantic Inadequacy)

- 測試工具使用**隨機值或字典**生成參數
- 缺乏語義真實性，導致大量 **4XX 錯誤**
- 隨機生成方法僅產生 **20% 有效 API 呼叫**
- 無法通過雲端閘道的語法/語義檢查

Problem

傳統 RESTful API 測試的核心問題

問題 2：序列長度不足 (Insufficient Sequence Length)

- 傳統工具生成的測試序列**過短**（1-2 步）
- 未考慮**參數必要性**與**操作依賴關係**
- 難以觸發**複雜操作組合**中的深層錯誤
- 無法執行跨資源依賴的操作鏈

Problem

傳統 RESTful API 測試的核心問題

問題 3：測試腳本問題 (Test Oracle Problem)

- 缺乏形式化規格或斷言，難以自動判定正確性
- End-to-End 驗證仍依賴**人工檢查**
- 多代理系統缺乏標準化通訊協定

Objective

核心目標

提出並驗證一套有效可落地的 RESTful API 黑箱測試自動化流程整合框架，提升測試正確性並降低維護成本。

- 1 提出 A2A 結合 MCP 的多協定混合架構**
結合水平代理協作（A2A）與垂直工具整合（MCP），明確分離語義推理與任務協調，提升代理協作的穩定性與可擴充性。
- 2 建構可自動化生成、執行與驗證的 RESTful API 測試流程**
利用 OpenAPI 標準、文件上下文與多代理分工，形成可 Re-loop 修正且語義一致的完整測試 Pipeline。
- 3 驗證混合式多代理架構在測試準確性的效果**
評估系統在生成測試腳本品質、測試結果一致性與工具協作效率上的優勢

Methodology

RESTLess 混合優化策略

語義增強 (Semantic Enhancement)

- 利用 LLM 生成 RTSet 資料集
- 有效請求序列提升 42.4%
- 解決參數語義不足問題

渲染順序優化

- 權重式參數渲染策略
- 必要參數優先渲染
- 錯誤偵測提升 54.7%

Methodology

多代理系統的演進

傳統框架的限制：

LangChain、AutoGen 將語義理解與流程協調混為一談，導致效能瓶頸

雙協定解決方案：

TB-CSPN 證明架構分離可提升效率 62.5%，LLM API 呼叫減少 66.7%

Methodology

雙協定混合架構 (Dual-Protocol Hybrid Architecture)

Model Context Protocol

- 垂直整合：LLM ↔ 工具
- 上下文一致性保證
- 安全的工具調用機制
- LangChain 實現

Agent-to-Agent Protocol

- 水平協作：Agent ↔ Agent
- 任務委派與狀態傳遞
- 跨主機通訊支援
- AutoGen 框架實現

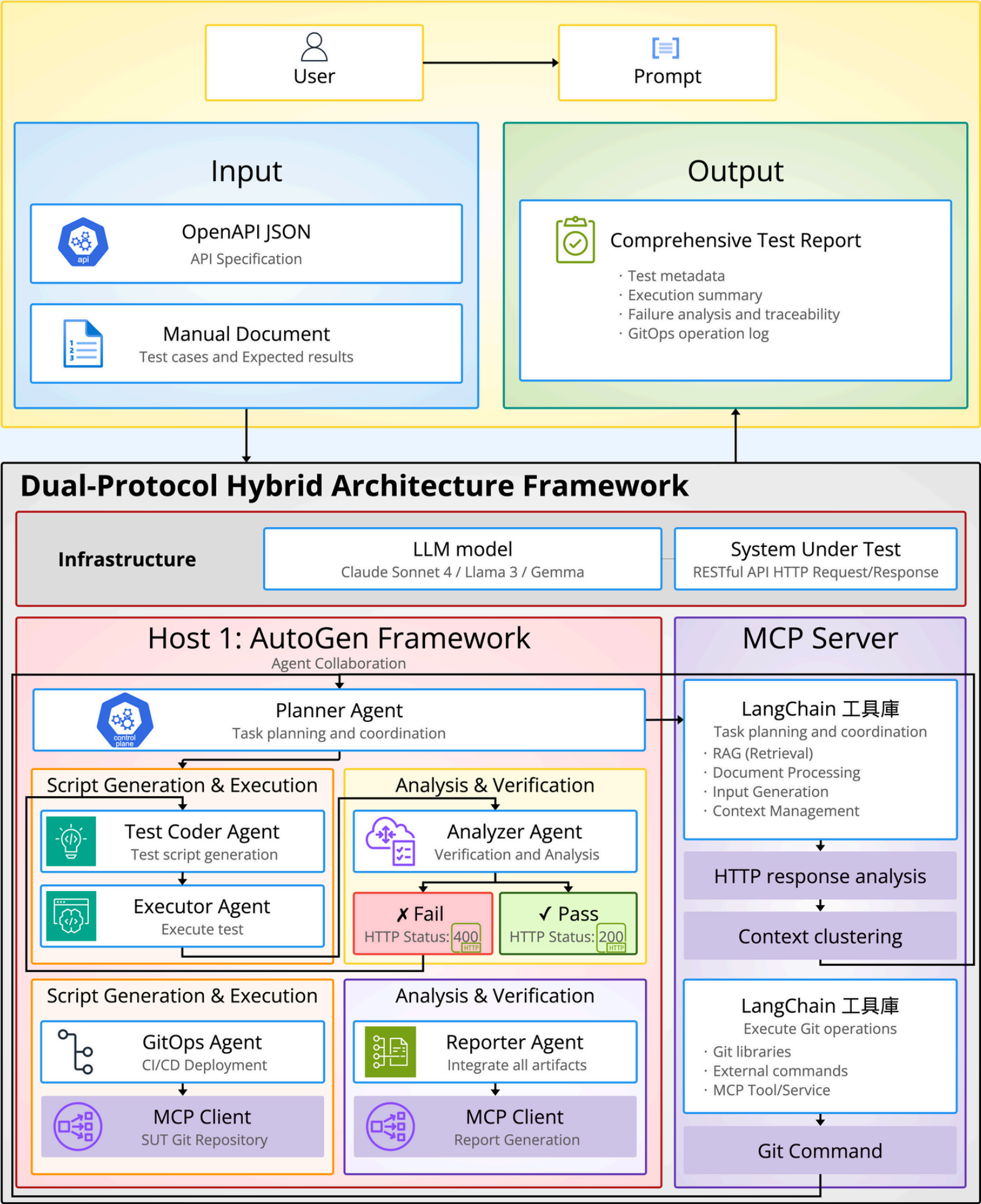
Methodology

雙協定混合架構 (Dual-Protocol Hybrid Architecture)

協作優勢

- 責任隔離：明確劃分代理通訊與工具使用
- 上下文一致性：MCP 確保長流程中的語義連貫
- 可擴展協作：跨主機、跨協議的標準化互操作
- 效能提升：避免集中式 LLM 協調瓶頸

System Architecture Diagram



前置處理：

MCP 語義增強、Core Context 建立

腳本生成：

A2A 任務委派、權重式渲染優化

結果分析：

自校準機制、Re-loop 修正

GitOps：

跨主機 A2A、CI/CD 整合

報告生成：

MCP 整合所有 Artifacts、結構化測試報告

系統架構：分散式雙主機設計

 Host 1: AutoGen A2A Host

角色：測試與分析主機

- Planner Agent - 協調
- Test Coder Agent - 生成
- Executor Agent - 執行
- Analyzer Agent - 分析
- Reporter Agent - 報告

 Host 2: GitOps Host

角色：部署與操作主機

- GitOps Agent - CI/CD 操作

執行操作：

- Git Tag 創建
- Branch Merge
- Pull Request 管理

安全隔離 + JWS 驗證

跨主機通訊：Host 1 (A2A Client) ↔ Host 2 (A2A Server)

關鍵創新技術

語義優化與上下文建立

- 輸入：OpenAPI JSON + Manual Document
- 處理：LLM 執行語義增強，生成 RTSet 參數值集合
- 輸出：語義增強後的規格 + 高語義 Core Context
- 效果：大幅提升測試序列通過雲端閘道的成功率

關鍵創新技術

Re-loop 自校準機制

- Analyzer Agent 判斷測試失敗時啟動
- 透過 A2A 回傳結構化失敗原因給 Planner
- MCP Context 維持狀態一致性
- Test Coder Agent 進行修復性代碼生成
- 循環直至測試成功，實現從失敗中學習

關鍵創新技術

權重式渲染順序優化

- 高權重參數 (Required) 優先渲染，確保有效性
- 機率 ω 決定是否加入低權重參數，增加多樣性
- 遵循 ODG (操作依賴圖) 順序要求
- 生成更長、更具跨資源依賴關係的測試序列

技術優勢與貢獻

- ✓ 自動化程度高：從規格輸入到報告生成全流程自動化
- ✓ 架構可擴展：雙主機設計 + 標準化協定
- ✓ 語義一致性：LLM 增強 + MCP 上下文保證
- ✓ CI/CD 整合：無縫銜接 GitOps 工作流程
- ✓ 自我修正能力：Re-loop 機制實現自主錯誤恢復
- ✓ 安全隔離：測試與部署環境權責分離

技術優勢與貢獻

學術貢獻

- ① 首次提出 **A2A + MCP** 雙協定整合應用於軟體測試領域
- ② 驗證混合架構在測試自動化中的效能與可行性
- ③ 提供可落地的多代理協作框架設計方法
- ④ 從測試生成到 CI/CD 部署的端到端自動化流程

Conclusion

研究總結

- 提出並設計雙協定混合架構的 RESTful API 自動化測試框架
- 整合 RESTLess 語義優化策略與多代理系統協作機制
- 實現從測試生成到 CI/CD 部署的端到端自動化流程
- 爲 LLM 驅動的軟體工程提供**實證研究基礎**
- 驗證 A2A 與 MCP 協定整合的實務可行性

Conclusion

未來研究方向

技術深化

- 形式化驗證方法整合
- 更多協定的互通性研究
- 效能優化與擴展性提升

應用擴展

- 微服務架構系統測試
- 企業級工作流程自動化
- 智能軟體工程生態系統



Thank You
for your attention!

Q & A